

Linux Command Benchmarking System

Course: System Programming and Scripting

Submitted by: The Beegs

Submitted to: Dr. Aditya Singh

Date: 7th December 2025

Introduction

The results of a system performance benchmark using a custom Bash shell script are presented in this report. This study's goal was to evaluate the device's processing power quantitatively, with a particular emphasis on disk I/O and CPU-intensive tasks. Standard Unix tools like grep, sort, tar, dd, and wc were used in the benchmark against large, artificially created data files of different sizes. The system's efficiency is directly measured by the resulting data, which culminates in a Benchmark Score that represents overall performance in relation to a specified reference time.

TEAM MEMBERS

- 1) Devansh (LIT2025036) - Benchmarking Commands & Team Management
- 2) Rohit(LCS2025052) - Global variable registry
- 3) Harshit(LCS2025034) - Clear Cache Function & Testing
- 4) Divit Pandey(LIT2025031) - Data Generation Function
- 5) Sreyash(LCS2025042) - Report Writing & Score Calculation
- 6) Daksh(LCS2025045) - Test Loop Iterations & Command testing
- 7) Aditya(LIT2025058) - Initialization
- 8) Aranya(LCS2025046) - Post Processing & Data management
- 9) Arshaan Baig(LCS2025053)- Results Calculation
- 10) Vaibhav(LIT2025056) - Documentation & Test files generation

Abstract

The purpose of our project is to find how some commonly used Linux commands—like **grep**, **sort**, **tar**, **dd**, and **wc**—work when they are given files of different sizes. Many people use these commands every day without thinking about how their performance changes with larger files. So, we wanted to explore that in a simple and clear way.

We tested each command with three different file sizes: **100 MB**, **200 MB**, and **500 MB**. By increasing the size step by step, we could clearly observe how the command's speed and system usage changed. For example, we looked at how long the command took to run and how the system reacted when handling bigger data. This helped us understand whether some commands slow down a lot with larger files or if they remain fairly stable. Our script also gives a score to the system performing these tasks.

To avoid manually repeating the same steps again and again, we created a small automated system. This system used scripts that did most of the work for us. It automatically generated the files, ran all the commands multiple times & collected time information. This made the whole process much easier and ensured that the results were more accurate and fair.

In the report, we describe how the system works, how we built the scripts, and how each test was carried out. We also present the results we got from the experiments and explain what they mean in simple terms. Along the way, we faced a few challenges—such as slow processing times for large files and ensuring that the measurements were correct—but we managed to overcome them.

Finally, we also talk about what improvements can be made in the future. This includes adding more tools, testing even larger files, or using better hardware to get deeper insights. Overall, the report provides a beginner-friendly explanation of how Linux commands behave with growing data sizes and what we learned from our analysis.

Objectives

- To test the given Linux tools across a range of file sizes.
- To evaluate performance using metrics like throughput and execution time.
- To use shell scripts to automate testing.
- To comprehend how command performance is affected by file size.

Modules Used

- Linux Utilities & Commands
 - Output & Error Redirection
 - echo, rm, mkdir, etc. (Basic Commands)
 - grep, wc, sort (Benchmarking Commands)
- Shell Scripting
 - Control Statements and Structures
 - Loops
 - Functions & Arguments
- File Handling and Networking
 - Reading & Writing Files

- Creation and modification of .csv & .txt files
- Permission Handling during production.

Implementation Details

- We used dd text generation scripts for file generation.
- For each file size, each command is executed 7 times.
- `/usr/bin/time -p` command used for time measurement.
- Stored logs in a structured format.
- Printed and stored the final performance score.

Important Shell Scripts

Generate files

```
dd if=/dev/urandom of="$FileName" bs=1M count="$CountMultiplayer"  
status=none;
```

Benchmarking grep command:

```
/usr/bin/time -f "$TemKaFormat" -o "$TimeFile" \  
    grep "$Pattern" "$FilePath" > /dev/null 2> /dev/null ;;
```

Benchmark sort

```
/usr/bin/time -f "$TemKaFormat" -o "$TimeFile" \  
    sort "$FilePath" > /dev/null ;;
```

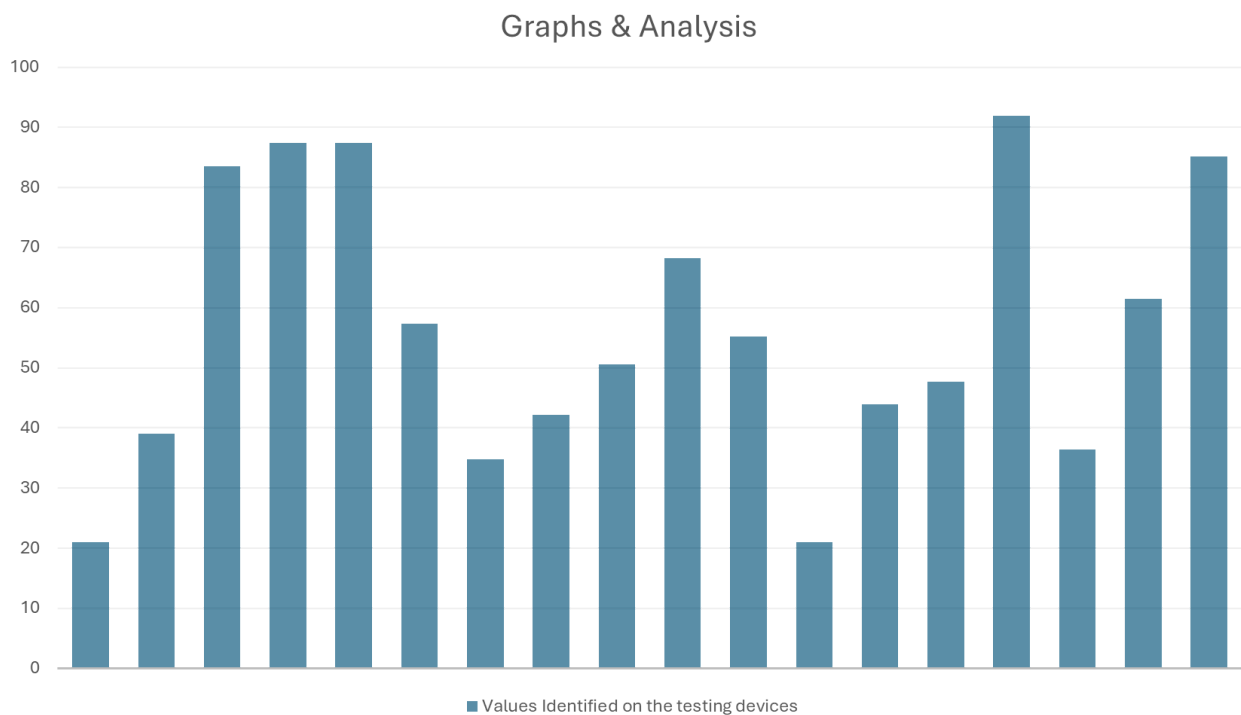
Benchmark tar

```
/usr/bin/time -f "$TemKaFormat" -o "$TimeFile" \  
    tar czf /dev/null "$FilePath" 2> /dev/null ;;
```

Benchmark wc

```
/usr/bin/time -f "$TemKaFormat" -o "$TimeFile" \  
    wc "$FilePath" > /dev/null ;;
```

Graphs, Metrics, and Observations



1. Macbooks
2. Windows PCs
 - a. Thin & Light Laptops
 - b. Gaming Laptops
 - c. Old Computers

Average Score: 60.31

Median: 59.42

Maximum Score: 91.98 (MacBook Air M4)

Minimum Score: 21.00 (Old HP Laptop)

Results and Discussion

Key findings:

1. grep performance is inversely related to file size. That means with an increase in file size, grep performance decreases, and the other way also holds true.
2. Due to high memory usage, sorting takes a lot of time to happen.
3. tar shows consistent performance based on compression type.
4. dd throughput depends heavily on block size.
5. wc is comparatively slowest of them all.

Challenges Faced

- Variation in system load affecting test accuracy.
- Ensuring fair benchmarking by clearing caches.
- Managing log files and automating data cleanup.
- Coming together as one and performing tasks.
- Conflicting ideas and options.

Conclusion

The benchmarking system successfully demonstrates performance variations among fundamental Linux utilities across different file sizes. The results highlight the efficiency and scalability of each command, providing valuable insight for system administrators, developers, and educators.

Future Scope

- Include CPU, memory, and I/O statistics.
- Use containerised benchmarking for consistent results.
- Web dashboard for real-time visualisation.

References

- Linux manual pages
- AI Models for help in debugging.
- Online Linux benchmarking blogs and articles of GFG.
- Shell scripting tutorial video of M Prashant (YouTube).